Advanced Python Programming Lab

Salva Soumya

23MID0061

5(a) Install OpenPyXL and Required Packages

In [68]:
```
#pip install openpyxl pandas matplotlib
```

Generate Sales Data Using a Dictionary

In [32]:
```
import openpyxl
import pandas as pd
import math

# Create sales data using a dictionary
sales_data = {
    'Product': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Printer', 'Tablet', 'Headset', 'Webcam', 'Speaker', 'Hard
    'Name': ['Dell Laptop', 'Logitech Mouse', 'HP Keyboard', 'Samsung Monitor', 'Canon Printer', 'Samsung Tablet', 'E
    'Region': ['South', 'North', 'East', 'West', 'South', 'North', 'East', 'West', 'South', 'North'],
    'Date': ['2026-01-01', '2026-01-02', '2026-01-03', '2026-01-04', '2026-01-05', '2026-01-06', '2026-01-07', '2026-
    'Sales': [5000, 800, 1500, 3500, 2200, 4500, 1200, 1800, 900, 3000]
}

# Convert dictionary into a DataFrame
initial_df = pd.DataFrame(sales_data)
print(initial_df)
```

	Product	Name	Region	Date	Sales
0	Laptop	Dell Laptop	South	2026-01-01	5000
1	Mouse	Logitech Mouse	North	2026-01-02	800
2	Keyboard	HP Keyboard	East	2026-01-03	1500
3	Monitor	Samsung Monitor	West	2026-01-04	3500
4	Printer	Canon Printer	South	2026-01-05	2200
5	Tablet	Samsung Tablet	North	2026-01-06	4500
6	Headset	Boat Headset	East	2026-01-07	1200
7	Webcam	Logitech Webcam	West	2026-01-08	1800
8	Speaker	JBL Speaker	South	2026-01-09	900
9	Hard Disk	Seagate HDD	North	2026-01-10	3000

In [33]:
```
# Save dictionary data to Excel
initial_df.to_excel('sales.xlsx', index=False, sheet_name='Sales')
print('sales.xlsx created successfully')
```

```
sales.xlsx created successfully
```

5(b) Load an Excel Workbook

In [34]:
```
from openpyxl import load_workbook

wb = load_workbook('sales.xlsx')
print('Workbook loaded successfully')
```

```
Workbook loaded successfully
```

5(c) Use the Active Attribute to Access the Active Worksheet

In [35]:
```
ws = wb.active
print('Active Worksheet:', ws.title)
```

```
Active Worksheet: Sales
```

5(d) Count the Number of Rows and Columns

In [36]:
```
print('Number of Rows:', ws.max_row)
print('Number of Columns:', ws.max_column)
```

```
Number of Rows: 11
Number of Columns: 5
```

5(e) Read Data from Specific Cell D1

In [37]:
```
print('Data in D1:', ws['D1'].value)
```

```
Data in D1: Date
```

5(f) Read Data in a Row from First Column to Nth Column

In [38]:
```
for col in range(1, ws.max_column + 1):
    print(ws.cell(row=1, column=col).value)
```

```
Product
Name
Region
Date
Sales
```

5(g) Assign a Value to a Cell Using Cell Name

In [39]:
```
ws['K1'] = 'Total Sales'
print('K1:', ws['K1'].value)
```

```
K1: Total Sales
```

5(h) Assign a Value Using Row and Column Positions

In [40]:
```
# Row 2, Column 10 is J2
ws.cell(row=2, column=10).value = 5000
print('J2:', ws.cell(row=2, column=10).value)
```

```
J2: 5000
```

5(i) Save the Excel File

In [41]:
```
wb.save('sales_updated.xlsx')
print('sales_updated.xlsx saved successfully')
```

```
sales_updated.xlsx saved successfully
```

5(j) Calculate the Sum of All Values in a Row and Assign the Total to a Cell

In [42]:
```
# Calculate the sum of Sales values from rows 2 to 11
total = 0
for row in range(2, ws.max_row + 1):
    value = ws.cell(row=row, column=5).value
    if isinstance(value, (int, float)):
        total += value

# Store the total in K2
ws['K1'] = 'Total Sales'
ws['K2'] = total
print('Total Sales:', ws['K2'].value)
```

```
Total Sales: 24400
```

5(k) Average Operation

In [43]:
```
values = []
for row in range(2, ws.max_row + 1):
    value = ws.cell(row=row, column=5).value
    if isinstance(value, (int, float)):
        values.append(value)

average = sum(values) / len(values)
ws['L1'] = 'Average Sales'
ws['L2'] = average
print('Average Sales:', average)
```

```
Average Sales: 2440.0
```

5(k) COUNTIF Operation

In [44]:
```
# Count sales greater than 1000
count = 0
for row in range(2, ws.max_row + 1):
    value = ws.cell(row=row, column=5).value
    if isinstance(value, (int, float)) and value > 1000:
        count += 1

ws['M1'] = 'Count > 1000'
ws['M2'] = count
print('Count of Sales > 1000:', count)
```

```
Count of Sales > 1000: 8
```

5(k) SUMIF Operation

In [45]:
```
# Sum sales greater than 1000
sumif = 0
for row in range(2, ws.max_row + 1):
    value = ws.cell(row=row, column=5).value
    if isinstance(value, (int, float)) and value > 1000:
        sumif += value

ws['N1'] = 'Sum > 1000'
ws['N2'] = sumif
print('SUMIF - Sales > 1000:', sumif)
```

```
SUMIF - Sales > 1000: 22700
```

5(k) CEILING Operation

In [46]:
```
# Round 1234 up to the nearest 100
number = 1234
ceiling_value = math.ceil(number / 100) * 100

ws['O1'] = 'Ceiling'
ws['O2'] = ceiling_value
print('CEILING:', ceiling_value)
```

```
CEILING: 1300
```

5(l) Rename a Sheet

In [47]:
```
ws.title = 'Sales_Data'
print('New Sheet Name:', ws.title)
```

```
New Sheet Name: Sales_Data
```

5(l) Duplicate a Sheet

In [48]:
```
copy_sheet = wb.copy_worksheet(ws)
copy_sheet.title = 'Sales_Copy'
print('Duplicated Sheet:', copy_sheet.title)
```

```
Duplicated Sheet: Sales_Copy
```

5(l) Delete a Sheet

In [49]:
```
wb.remove(copy_sheet)
print('Sales_Copy sheet deleted')
```

```
Sales_Copy sheet deleted
```

5(m) Add a Bar Chart to an Excel Sheet

In [50]:
```
from openpyxl.chart import BarChart, Reference

# Create a BarChart object
chart = BarChart()
chart.title = 'Sales by Product'
chart.x_axis.title = 'Product'
chart.y_axis.title = 'Sales'

# Set minimum column, maximum column, minimum row and maximum row for sales data
# Sales is in column E
data = Reference(ws, min_col=5, max_col=5, min_row=1, max_row=ws.max_row)

# Set parameters for chart categories
```

```python
# Product is in column A
categories = Reference(ws, min_col=1, max_col=1, min_row=2, max_row=ws.max_row)

# Add data to chart
chart.add_data(data, titles_from_data=True)

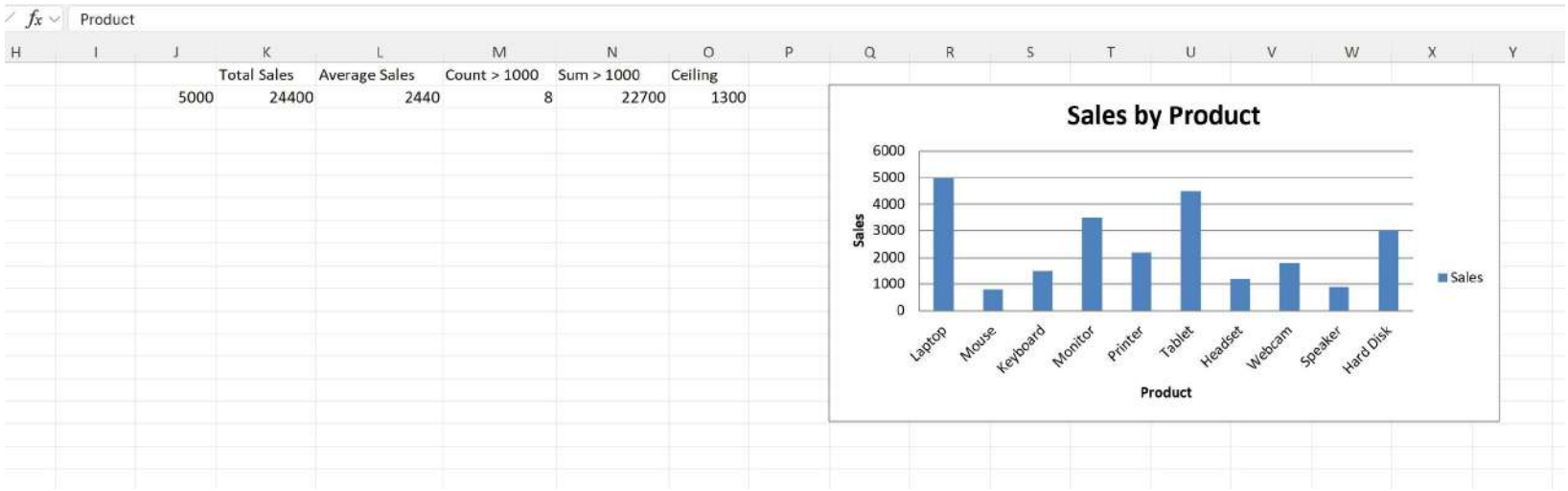
# Set categories
chart.set_categories(categories)

# Add chart object to worksheet
ws.add_chart(chart, 'Q2')

# Save Excel file
wb.save('sales_final.xlsx')
print('Bar chart added and sales_final.xlsx saved successfully')
```

```
Bar chart added and sales_final.xlsx saved successfully
```

OUTPUT SCREESHOT



6. Using Pandas

6(a) Load the Excel Sheet to a DataFrame

In [51]:
```
df = pd.read_excel('sales_final.xlsx', sheet_name='Sales_Data')
print('Excel sheet loaded into DataFrame')
print(df)
```

```
Excel sheet loaded into DataFrame
     Product             Name Region        Date  Sales  Unnamed: 5  \
0     Laptop      Dell Laptop  South  2026-01-01   5000         NaN
1      Mouse   Logitech Mouse  North  2026-01-02    800         NaN
2   Keyboard      HP Keyboard   East  2026-01-03   1500         NaN
3    Monitor  Samsung Monitor   West  2026-01-04   3500         NaN
4    Printer    Canon Printer  South  2026-01-05   2200         NaN
5     Tablet   Samsung Tablet  North  2026-01-06   4500         NaN
6    Headset     Boat Headset   East  2026-01-07   1200         NaN
7     Webcam  Logitech Webcam   West  2026-01-08   1800         NaN
8    Speaker      JBL Speaker  South  2026-01-09    900         NaN
9  Hard Disk      Seagate HDD  North  2026-01-10   3000         NaN

   Unnamed: 6  Unnamed: 7  Unnamed: 8  Unnamed: 9  Total Sales  Average Sales  \
0         NaN         NaN         NaN      5000.0      24400.0         2440.0
1         NaN         NaN         NaN         NaN          NaN            NaN
2         NaN         NaN         NaN         NaN          NaN            NaN
3         NaN         NaN         NaN         NaN          NaN            NaN
4         NaN         NaN         NaN         NaN          NaN            NaN
5         NaN         NaN         NaN         NaN          NaN            NaN
6         NaN         NaN         NaN         NaN          NaN            NaN
7         NaN         NaN         NaN         NaN          NaN            NaN
8         NaN         NaN         NaN         NaN          NaN            NaN
9         NaN         NaN         NaN         NaN          NaN            NaN

   Count > 1000  Sum > 1000  Ceiling
0           8.0     22700.0   1300.0
1           NaN         NaN      NaN
2           NaN         NaN      NaN
3           NaN         NaN      NaN
4           NaN         NaN      NaN
5           NaN         NaN      NaN
6           NaN         NaN      NaN
7           NaN         NaN      NaN
8           NaN         NaN      NaN
9           NaN         NaN      NaN
```

6(b) Inspect the Data using head(), info() and describe()

In [52]:
```
print('First 5 Rows:')
print(df.head())
print('\nDataFrame Information:')
df.info()
print('\nStatistical Description:')
print(df.describe())
```

```
First 5 Rows:
    Product             Name Region        Date  Sales  Unnamed: 5  \
0    Laptop      Dell Laptop  South  2026-01-01   5000         NaN
1     Mouse   Logitech Mouse  North  2026-01-02    800         NaN
2  Keyboard      HP Keyboard   East  2026-01-03   1500         NaN
3   Monitor  Samsung Monitor   West  2026-01-04   3500         NaN
4   Printer    Canon Printer  South  2026-01-05   2200         NaN

   Unnamed: 6  Unnamed: 7  Unnamed: 8  Unnamed: 9  Total Sales  Average Sales  \
0         NaN         NaN         NaN      5000.0      24400.0         2440.0
1         NaN         NaN         NaN         NaN          NaN            NaN
2         NaN         NaN         NaN         NaN          NaN            NaN
3         NaN         NaN         NaN         NaN          NaN            NaN
4         NaN         NaN         NaN         NaN          NaN            NaN

   Count > 1000  Sum > 1000  Ceiling
0           8.0     22700.0   1300.0
1           NaN         NaN      NaN
2           NaN         NaN      NaN
3           NaN         NaN      NaN
4           NaN         NaN      NaN

DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 15 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Product        10 non-null     object
 1   Name           10 non-null     object
 2   Region         10 non-null     object
 3   Date           10 non-null     object
 4   Sales          10 non-null     int64
 5   Unnamed: 5     0 non-null      float64
 6   Unnamed: 6     0 non-null      float64
 7   Unnamed: 7     0 non-null      float64
 8   Unnamed: 8     0 non-null      float64
 9   Unnamed: 9     1 non-null      float64
 10  Total Sales    1 non-null      float64
 11  Average Sales  1 non-null      float64
 12  Count > 1000   1 non-null      float64
 13  Sum > 1000     1 non-null      float64
```

```
 14  Ceiling        1 non-null      float64
dtypes: float64(10), int64(1), object(4)
memory usage: 1.3+ KB

Statistical Description:
             Sales  Unnamed: 5  Unnamed: 6  Unnamed: 7  Unnamed: 8  \
count    10.000000         0.0         0.0         0.0         0.0
mean   2440.000000         NaN         NaN         NaN         NaN
std    1497.553561         NaN         NaN         NaN         NaN
min     800.000000         NaN         NaN         NaN         NaN
25%    1275.000000         NaN         NaN         NaN         NaN
50%    2000.000000         NaN         NaN         NaN         NaN
75%    3375.000000         NaN         NaN         NaN         NaN
max    5000.000000         NaN         NaN         NaN         NaN

       Unnamed: 9  Total Sales  Average Sales  Count > 1000  Sum > 1000  \
count         1.0          1.0            1.0           1.0         1.0
mean       5000.0      24400.0         2440.0           8.0     22700.0
std           NaN          NaN            NaN           NaN         NaN
min        5000.0      24400.0         2440.0           8.0     22700.0
25%        5000.0      24400.0         2440.0           8.0     22700.0
50%        5000.0      24400.0         2440.0           8.0     22700.0
75%        5000.0      24400.0         2440.0           8.0     22700.0
max        5000.0      24400.0         2440.0           8.0     22700.0

       Ceiling
count      1.0
mean    1300.0
std        NaN
min     1300.0
25%     1300.0
50%     1300.0
75%     1300.0
max     1300.0
```

6(c) Print Data by a Specific Column Name

In [53]:
```
print(df['Sales'])
```

```
0    5000
1     800
2    1500
3    3500
4    2200
5    4500
6    1200
7    1800
8     900
9    3000
Name: Sales, dtype: int64
```

6(d) Print Data from the First Five Rows

In [54]:
```
print(df.head(5))
```

```
    Product             Name Region        Date  Sales  Unnamed: 5  \
0    Laptop      Dell Laptop  South  2026-01-01   5000         NaN
1     Mouse   Logitech Mouse  North  2026-01-02    800         NaN
2  Keyboard      HP Keyboard   East  2026-01-03   1500         NaN
3   Monitor  Samsung Monitor   West  2026-01-04   3500         NaN
4   Printer    Canon Printer  South  2026-01-05   2200         NaN

   Unnamed: 6  Unnamed: 7  Unnamed: 8  Unnamed: 9  Total Sales  Average Sales  \
0         NaN         NaN         NaN      5000.0      24400.0         2440.0
1         NaN         NaN         NaN         NaN          NaN            NaN
2         NaN         NaN         NaN         NaN          NaN            NaN
3         NaN         NaN         NaN         NaN          NaN            NaN
4         NaN         NaN         NaN         NaN          NaN            NaN

   Count > 1000  Sum > 1000  Ceiling
0           8.0     22700.0   1300.0
1           NaN         NaN      NaN
2           NaN         NaN      NaN
3           NaN         NaN      NaN
4           NaN         NaN      NaN
```

6(e) Filter Data in a Specific Column with a Specific Condition

In [55]:
```
# Display rows where Sales is greater than 1000
filtered_data = df[df['Sales'] > 1000]
print(filtered_data)
```

```
    Product             Name Region        Date  Sales  Unnamed: 5  \
0    Laptop      Dell Laptop  South  2026-01-01   5000         NaN
2  Keyboard      HP Keyboard   East  2026-01-03   1500         NaN
3   Monitor  Samsung Monitor   West  2026-01-04   3500         NaN
4   Printer    Canon Printer  South  2026-01-05   2200         NaN
5    Tablet   Samsung Tablet  North  2026-01-06   4500         NaN
6   Headset     Boat Headset   East  2026-01-07   1200         NaN
7    Webcam  Logitech Webcam   West  2026-01-08   1800         NaN
9  Hard Disk      Seagate HDD  North  2026-01-10   3000         NaN

   Unnamed: 6  Unnamed: 7  Unnamed: 8  Unnamed: 9  Total Sales  Average Sales  \
0         NaN         NaN         NaN      5000.0      24400.0         2440.0
2         NaN         NaN         NaN         NaN          NaN            NaN
3         NaN         NaN         NaN         NaN          NaN            NaN
4         NaN         NaN         NaN         NaN          NaN            NaN
5         NaN         NaN         NaN         NaN          NaN            NaN
6         NaN         NaN         NaN         NaN          NaN            NaN
7         NaN         NaN         NaN         NaN          NaN            NaN
9         NaN         NaN         NaN         NaN          NaN            NaN

   Count > 1000  Sum > 1000  Ceiling
0           8.0     22700.0   1300.0
2           NaN         NaN      NaN
3           NaN         NaN      NaN
4           NaN         NaN      NaN
5           NaN         NaN      NaN
6           NaN         NaN      NaN
7           NaN         NaN      NaN
9           NaN         NaN      NaN
```

6(f) Add a New Column Based on a Condition

In [56]:
```
# Create Sales_Category based on Sales value
df['Sales_Category'] = df['Sales'].apply(lambda x: 'High' if x > 1000 else 'Low')
print(df)
```

	Product	Name	Region	Date	Sales	Unnamed: 5	\
0	Laptop	Dell Laptop	South	2026-01-01	5000	NaN	
1	Mouse	Logitech Mouse	North	2026-01-02	800	NaN	
2	Keyboard	HP Keyboard	East	2026-01-03	1500	NaN	
3	Monitor	Samsung Monitor	West	2026-01-04	3500	NaN	
4	Printer	Canon Printer	South	2026-01-05	2200	NaN	
5	Tablet	Samsung Tablet	North	2026-01-06	4500	NaN	
6	Headset	Boat Headset	East	2026-01-07	1200	NaN	
7	Webcam	Logitech Webcam	West	2026-01-08	1800	NaN	
8	Speaker	JBL Speaker	South	2026-01-09	900	NaN	
9	Hard Disk	Seagate HDD	North	2026-01-10	3000	NaN	

	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Total Sales	Average Sales	\
0	NaN	NaN	NaN	5000.0	24400.0	2440.0	
1	NaN	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN	NaN	NaN	NaN	
6	NaN	NaN	NaN	NaN	NaN	NaN	
7	NaN	NaN	NaN	NaN	NaN	NaN	
8	NaN	NaN	NaN	NaN	NaN	NaN	
9	NaN	NaN	NaN	NaN	NaN	NaN	

	Count > 1000	Sum > 1000	Ceiling	Sales_Category
0	8.0	22700.0	1300.0	High
1	NaN	NaN	NaN	Low
2	NaN	NaN	NaN	High
3	NaN	NaN	NaN	High
4	NaN	NaN	NaN	High
5	NaN	NaN	NaN	High
6	NaN	NaN	NaN	High
7	NaN	NaN	NaN	High
8	NaN	NaN	NaN	Low
9	NaN	NaN	NaN	High

6(g) Update a Value

In [57]:
```
# Update the Sales value in the first row
df.loc[0, 'Sales'] = 5500
```

```
print(df.loc[0])
```

```
Product               Laptop
Name              Dell Laptop
Region                  South
Date                2026-01-01
Sales                     5500
Unnamed: 5                 NaN
Unnamed: 6                 NaN
Unnamed: 7                 NaN
Unnamed: 8                 NaN
Unnamed: 9              5000.0
Total Sales            24400.0
Average Sales           2440.0
Count > 1000               8.0
Sum > 1000             22700.0
Ceiling                 1300.0
Sales_Category            High
Name: 0, dtype: object
```

6(h) Print the DataFrame

In [58]:
```
print(df)
```

	Product	Name	Region	Date	Sales	Unnamed: 5	\
0	Laptop	Dell Laptop	South	2026-01-01	5500	NaN	
1	Mouse	Logitech Mouse	North	2026-01-02	800	NaN	
2	Keyboard	HP Keyboard	East	2026-01-03	1500	NaN	
3	Monitor	Samsung Monitor	West	2026-01-04	3500	NaN	
4	Printer	Canon Printer	South	2026-01-05	2200	NaN	
5	Tablet	Samsung Tablet	North	2026-01-06	4500	NaN	
6	Headset	Boat Headset	East	2026-01-07	1200	NaN	
7	Webcam	Logitech Webcam	West	2026-01-08	1800	NaN	
8	Speaker	JBL Speaker	South	2026-01-09	900	NaN	
9	Hard Disk	Seagate HDD	North	2026-01-10	3000	NaN	

	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Total Sales	Average Sales	\
0	NaN	NaN	NaN	5000.0	24400.0	2440.0	
1	NaN	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN	NaN	NaN	NaN	
6	NaN	NaN	NaN	NaN	NaN	NaN	
7	NaN	NaN	NaN	NaN	NaN	NaN	
8	NaN	NaN	NaN	NaN	NaN	NaN	
9	NaN	NaN	NaN	NaN	NaN	NaN	

	Count > 1000	Sum > 1000	Ceiling	Sales_Category
0	8.0	22700.0	1300.0	High
1	NaN	NaN	NaN	Low
2	NaN	NaN	NaN	High
3	NaN	NaN	NaN	High
4	NaN	NaN	NaN	High
5	NaN	NaN	NaN	High
6	NaN	NaN	NaN	High
7	NaN	NaN	NaN	High
8	NaN	NaN	NaN	Low
9	NaN	NaN	NaN	High

6(i) Load the Updated DataFrame Back to Excel

In [59]:
```
df.to_excel('sales_pandas_updated.xlsx', index=False, sheet_name='Sales_Data')
print('Updated DataFrame saved to sales_pandas_updated.xlsx')
```

```
Updated DataFrame saved to sales_pandas_updated.xlsx
```

6(j) Display and Save a Screenshot of the Updated Excel Sheet

In [66]:
```python
# 6(j). Save and display the updated Excel sheet

from IPython.display import display, Image
import pandas as pd

# Save the updated DataFrame to Excel
df.to_excel("sales_pandas_updated.xlsx", index=False, sheet_name="Sales_Data")

print("Excel file saved successfully.")

# Display the updated DataFrame in Jupyter
display(df)
```

```
Excel file saved successfully.
```

	Product	Name	Region	Date	Sales	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Total Sales	Average Sales	Count > 1000
0	Laptop	Dell Laptop	South	2026-01-01	5500	NaN	NaN	NaN	NaN	5000.0	24400.0	2440.0	8.0
1	Mouse	Logitech Mouse	North	2026-01-02	800	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	Keyboard	HP Keyboard	East	2026-01-03	1500	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	Monitor	Samsung Monitor	West	2026-01-04	3500	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	Printer	Canon Printer	South	2026-01-05	2200	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	Tablet	Samsung Tablet	North	2026-01-06	4500	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	Headset	Boat Headset	East	2026-01-07	1200	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	Webcam	Logitech Webcam	West	2026-01-08	1800	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
8	Speaker	JBL Speaker	South	2026-01-09	900	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	Hard Disk	Seagate HDD	North	2026-01-10	3000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN



Final files:



sales	07-09-2026 14:22	Microsoft Excel Work...	6 KB
sales_final	07-09-2026 14:22	Microsoft Excel Work...	7 KB
sales_pandas_updated	07-09-2026 14:38	Microsoft Excel Work...	6 KB
sales_updated	07-09-2026 14:22	Microsoft Excel Work...	6 KB
updated_excel_screenshot	07-09-2026 14:22	PNG File	62 KB

In [ ]: